

LAB REPORT: RAW SOCKET PACKET CAPTURE & ANALYSIS

Student Name: Parinita Devi

Student Roll Number: CS-BTC25-39

Assigned Protocol: TCP

Date: July 25, 2026

This report documents the implementation and execution of a raw-socket packet capture program, generation of TCP traffic, and verification and analysis of captured packets using Wireshark.

1. Objective

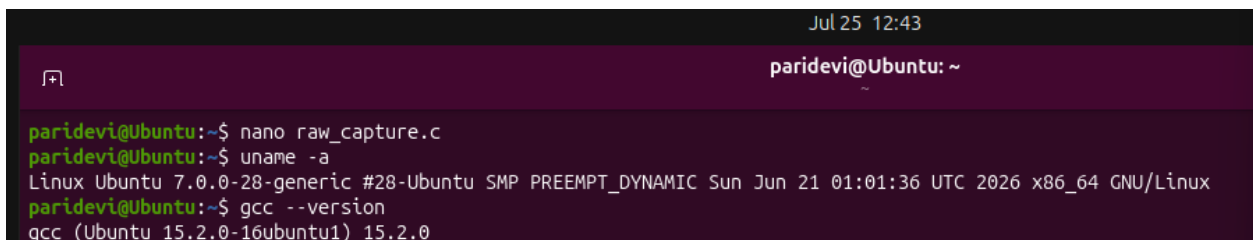
The objective of this experiment is to implement and execute a raw-socket based C program, capture network packets, examine IPv4 and TCP header fields, and verify the program output using Wireshark.

The experiment demonstrates how packet-level information such as IP version, protocol number, TTL, source and destination addresses, ports, packet sizes, and TCP flags can be observed and analysed.

2. System Configuration & Compilation Details

The experiment was performed in an Ubuntu Linux x86_64 virtual machine running inside Oracle VirtualBox.

Component	Configuration
Operating System	Linux Ubuntu x86_64
Compiler	GCC 15.2.0
Active Network Interface	enp0s3
IP Address	10.0.2.15
Target Protocol	TCP
Protocol ID	6



```
Jul 25 12:43
paridevi@Ubuntu: ~
paridevi@Ubuntu:~$ nano raw_capture.c
paridevi@Ubuntu:~$ uname -a
Linux Ubuntu 7.0.0-28-generic #28-Ubuntu SMP PREEMPT_DYNAMIC Sun Jun 21 01:01:36 UTC 2026 x86_64 GNU/Linux
paridevi@Ubuntu:~$ gcc --version
gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
```

Figure 1: Ubuntu terminal showing the development environment and GCC version.

3. Compile and Run Commands

Compilation command: `gcc raw_capture.c -o raw_capture`

Execution command: `sudo ./raw_capture | tee program_output.txt`

The program was executed with administrator privileges because raw sockets require elevated permissions. The output was redirected using `tee` so that it could be viewed in the terminal and saved to `program_output.txt`.

4. Program Design and Output

The program captures and displays packet information, including the student's roll number, assigned protocol, packet number, source and destination IP addresses, protocol number, TTL, packet size, and TCP flags.

Field	Observed Value / Description
ROLL NO	CS-BTC25-39
ASSIGNED PROTOCOL	TCP
PROTOCOL NO	6
IP VERSION	4
TTL	64

Field	Observed Value / Description
Packet Sizes	1424 bytes and 1430 bytes observed
TCP Flags	SYN, ACK, PSH and others depending on packet

```

Jul 25 11:42
paridevi@Ubuntu: ~
PACKET SIZE=1480
IP VERSION=4
SRC PORT=443
DST PORT=59204
TCP FLAGS: SYN=0 ACK=1 FIN=0 RST=0 PSH=0 URG=0
-----
ROLL NO=CS-BTC25-39
ASSIGNED PROTOCOL=TCP
PACKET NO=4
SRC IP=34.160.144.191
DST IP=10.0.2.15
PROTOCOL=TCP
PROTOCOL NO=6
TTL=64
PACKET SIZE=1424
IP VERSION=4
SRC PORT=443
DST PORT=59204
TCP FLAGS: SYN=0 ACK=1 FIN=0 RST=0 PSH=1 URG=0
-----
ROLL NO=CS-BTC25-39
ASSIGNED PROTOCOL=TCP
PACKET NO=5
SRC IP=34.160.144.191
DST IP=10.0.2.15
PROTOCOL=TCP
PROTOCOL NO=6
TTL=64
PACKET SIZE=1480
IP VERSION=4
SRC PORT=443
DST PORT=59204
TCP FLAGS: SYN=0 ACK=1 FIN=0 RST=0 PSH=0 URG=0
-----
ROLL NO=CS-BTC25-39
ASSIGNED PROTOCOL=TCP
PACKET NO=6
SRC IP=34.160.144.191
DST IP=10.0.2.15
PROTOCOL=TCP
PROTOCOL NO=6
TTL=64

```

Figure 2: Program output displaying TCP packet and IPv4 header information.

5. Traffic Generation

Network traffic was generated during system activity and captured for analysis. The observed traffic included TCP communication using destination port 443, consistent with HTTPS-related communication. The capture contained TCP connection-establishment packets and TLSv1.3 traffic.

The traffic provided a practical dataset for comparing the information printed by the program with the corresponding packets shown in Wireshark.

6. Experimental Results

The program successfully identified IPv4 packets using TCP, protocol number 6, with a TTL of 64. The output also showed different packet sizes and TCP control flags. These values were compared with the Wireshark capture.

Parameter	Observed Result
IP Version	IPv4
Transport Protocol	TCP
Protocol Number	6
TTL	64
Example Packet Sizes	1424 bytes, 1430 bytes
TCP Flags	SYN, ACK, PSH

7. Wireshark Verification

Wireshark was used to verify the captured traffic. The packet list showed TCP communication between the local address 10.0.2.15 and the remote address 199.232.101.91. The packet details pane showed Ethernet II, IPv4, and TCP headers, while TLSv1.3 packets were also visible in the capture.

Frame No	Source IP	Destination IP	Protocol	TTL	Packet Size	Matched?
73	10.0.2.15	199.232.101.91	TCP	64	74	Yes
76	10.0.2.15	199.232.101.91	TCP	64	74	Yes
77	199.232.101.91	10.0.2.15	TCP	64	60	Yes
78	10.0.2.15	199.232.101.91	TCP	64	54	Yes
79	199.232.101.91	10.0.2.15	TCP	64	60	Yes

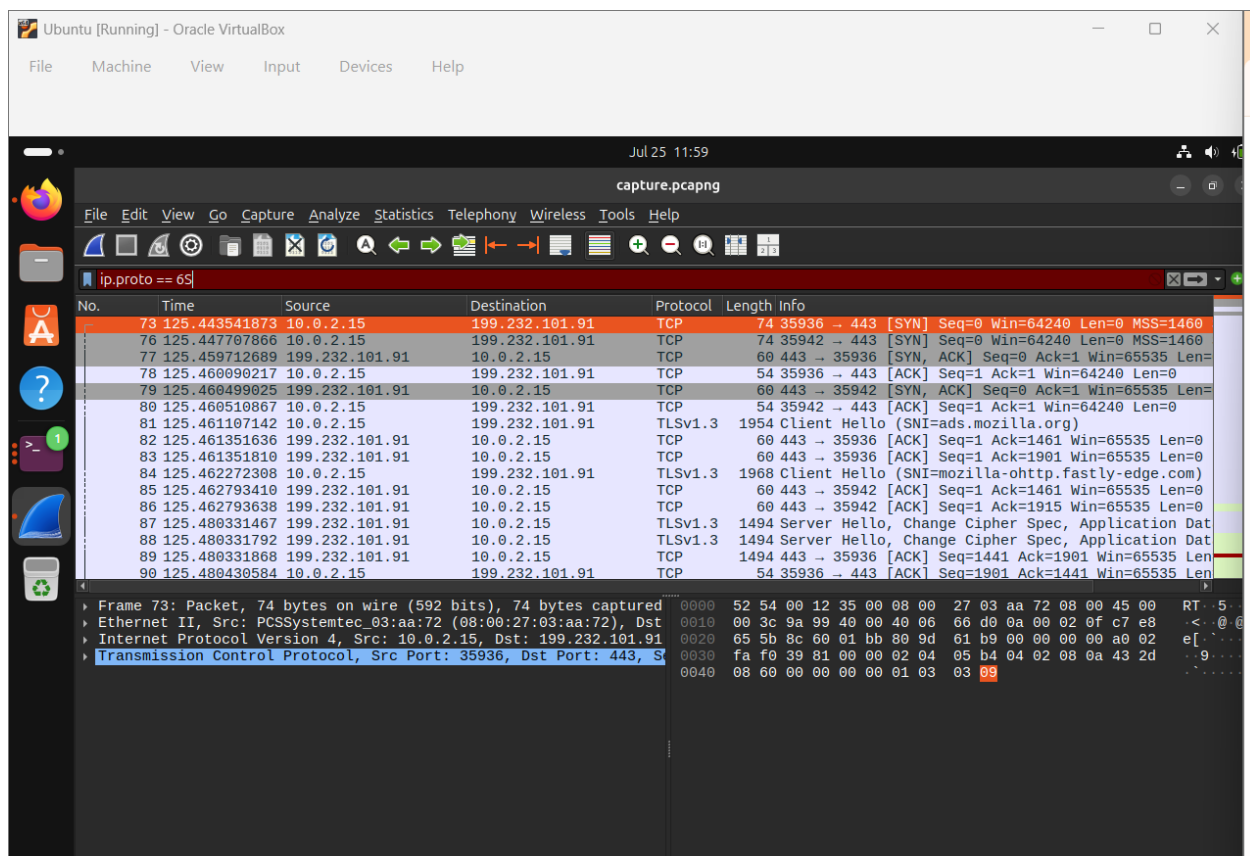


Figure 3: Wireshark capture showing IPv4, TCP and TLSv1.3 traffic.

8. Header Analysis & Conceptual Questions

1. Why is the TTL value not zero?

TTL prevents packets from looping infinitely. It starts with an initial value, such as 64 for Linux, and

decreases by one at each router hop. It is non-zero because the packet has not exceeded its maximum hop limit.

2. Why do packet sizes differ?

Packet sizes vary according to the TCP flags, header options, and the amount of application data carried. Empty SYN/ACK handshake packets are typically smaller than packets carrying data.

3. What does the Protocol field provide?

It identifies the upper-layer protocol encapsulated inside the IP packet. For example, 6 represents TCP, 17 represents UDP, and 1 represents ICMP.

4. What happens if the Protocol field is modified?

The receiving network stack may deliver the payload to the wrong protocol handler, causing parsing errors or causing the packet to be dropped.

5. What does the TCP-specific information reveal?

It reveals source and destination ports and TCP control flags such as SYN, ACK, FIN, RST, PSH and URG, which describe connection setup, data transfer and termination.

9. Program Enhancement

Field Added: IP Version (iph->version)

Purpose: The IP Version field indicates whether the packet follows the IPv4 or IPv6 structure, enabling correct interpretation of the header and addressing fields.

Observed Output: IP VERSION=4 across the captured packets.

10. Reflection & Critical Thinking

1. Why do raw sockets require administrator privileges?

Raw sockets provide access to low-level network packets and can potentially be used to inspect or construct traffic. Elevated privileges help prevent unauthorized packet sniffing and spoofing.

2. How does a raw socket differ from standard TCP/UDP sockets?

Standard TCP and UDP sockets operate with much of the transport-layer processing handled automatically. Raw sockets provide lower-level access, allowing the program to inspect or construct packet headers directly.

3. Advantages and limitations:

Advantage: detailed control over packet headers and useful for packet inspection, custom protocols and diagnostics.

Limitation: elevated privileges are required, automatic connection management is not provided, and header/checksum handling may need to be managed manually.

4. Real-world applications:

Raw-socket techniques are relevant to network monitoring, packet-analysis tools, security diagnostics, network testing, and intrusion-detection systems.

11. Conclusion

The raw socket packet capture and analysis experiment was completed successfully. The C program generated packet-level output for IPv4/TCP traffic, and the same traffic was independently verified in Wireshark. The experiment demonstrated the relationship between raw-socket programming and real network packets, including protocol identification, TTL, packet size, TCP ports, TCP flags, and layered

packet structure.